TalkLate

Traduccion en tiempo real con conciencia cultural y emocional

11. Decisiones de arquitectura (ADR)

Trabajo de grado

Julian David Diaz Medina

Diego Alejandro Franco Atehortua

Director: Wider Farid Sanchez Garzon

Version 1.1 - Noviembre de 2026

Fase 3 - Arquitectura de software y base de datos

Version: 1.0 (29/10/2026)

Metodo: las decisiones se validaron construyendo una prueba de concepto (14/10/2026) y se sustentan aqui con su contexto, las alternativas evaluadas y sus consecuencias. Cada ADR responde a uno o mas drivers (*documento 6 (Drivers de arquitectura)*); los patrones citados vienen del catalogo de patrones arquitectonicos del proyecto.

ADR	Decision	Patron del catalogo	Estado
ADR-001	App movil con Expo (React Native) y NativeWind	-	aceptada
ADR-002	Backend monolitico en Node.js con Express, API REST sincrona	Request-Reply (2.3)	aceptada
ADR-003	SQLite como base de datos inicial	-	reemplazada por ADR-012 (queda para pruebas sin servidor)
ADR-004	Traduccion con Qwen local, Claude Sonnet de respaldo y motor de reglas	Circuit Breaker (4.1) con degradacion aceptable	aceptada
ADR-005	Voz: dictado en el cliente y expo-speech para la salida	-	reemplazada en la entrada por ADR-014 y en la salida por ADR-015 (expo-speech queda de respaldo)
ADR-006	Autenticacion con JWT de acceso corto y token de refresco rotativo	Token-Based Authentication (5.1)	aceptada
ADR-007	Interfaz basada en el template Luna	-	aceptada
ADR-008	Ajustes al modelo de dominio al implementarlo	-	aceptada
ADR-009	Identificadores del dominio en espanol (lenguaje ubicuo)	-	aceptada (excepcion: esquema de SurrealDB en ingles, ver ADR-012)
ADR-010	Proteccion de datos personales	Structured Logging (6.1)	aceptada
ADR-011	API v1 segun la convencion REST y documentada con OpenAPI	Request-Reply (2.3)	aceptada
ADR-012	SurrealDB como fuente de verdad, detras de un repositorio (esquema en ingles, instancia remota)	Repository	aceptada (PostgreSQL evaluado y descartado)
ADR-013	Arquitectura hexagonal (puertos y adaptadores) dentro del monolito modular	Puertos y adaptadores	aceptada

ADR	Decision	Patron del catalogo	Estado
ADR-014	Voz en Expo Go: grabacion en el celular, Whisper en el servidor y entonacion por texto y voz	-	aceptada
ADR-015	Voz natural con emocion en el servidor (puerto Sintetizador)	Cadena de respaldo	aceptada
ADR-016	VoiceStudio local como primer motor de voz con emocion	-	aceptada
ADR-017	Piramide de pruebas en tres niveles, todos en el CI	-	aceptada
ADR-018	Escaneo de seguridad con Trivy en el CI	-	aceptada

Patrones del catalogo que **no** se usan y por que: Database per Service, API Gateway y BFF (un solo cliente y un solo backend; el catalogo lo indica asi para el MVP), Strangler Fig (no hay sistema legado), CQRS y Event Sourcing (el historial es un CRUD simple; no hay requisito de auditoria completa). Se revisaran si crece el volumen o aparecen nuevos clientes.

ADR-001: App movil con Expo (React Native) y NativeWind

Fecha: 2026-10-14 · **Estado:** aceptada · **Responde a:** DRV-07

Contexto

Dos personas desarrollan; la app debe servir en iOS y Android; el template de interfaz elegido (Luna) esta hecho en Expo.

Decision

Expo con React Native, expo-router para la navegacion y NativeWind (Tailwind) para los estilos. Se empezo con el SDK 54 y el 15/10/2026 se paso al SDK 57 para que la app abra en la version actual de Expo Go (PR #31). La misma base de codigo corre en web, lo que permite probar sin celular.

Alternativas (comparadas al justificar)

- **Flutter:** una sola base de codigo tambien, pero obliga a aprender Dart y no aprovecha el template ya comprado.
- **Nativo (Swift y Kotlin):** mejor acceso al dispositivo, pero son dos desarrollos para un equipo de dos.
- **Web responsiva (PWA):** mas simple, pero el microfono, la voz y la experiencia movil son mas limitados.

Consecuencias

Se prueba en Expo Go sin compilar. El reconocimiento de voz nativo y las notificaciones pueden requerir un "development build" mas adelante.

ADR-002: Backend monolitico en Node.js con Express, API REST sincrona

Fecha: 2026-10-14 · **Estado:** aceptada · **Responde a:** DRV-07, DRV-08 · **Patron:** Request-Reply

Contexto

Un solo cliente (la app) y un equipo pequeno. La traduccion es "bajo demanda con respuesta inmediata esperada por el usuario", el caso de uso que el catalogo asigna a Request-Reply.

Decision

Monolito modular en Node.js 22 con Express: rutas por recurso y reglas separadas de HTTP (desde ADR-013, en capas hexagonales: `src/dominio`, `src/aplicacion`, `src/adaptadores`), API REST sincrona. Mismo lenguaje (JavaScript/TypeScript) que la app.

Alternativas (comparadas al justificar)

- **Microservicios** (traduccion aparte): el catalogo los reserva para despliegue y escalado independientes; aqui solo agregan complejidad.
- **WebSocket o streaming:** el catalogo lo menciona para tiempo real estricto; como la traduccion es por turno (documento 1, Vision y contexto), REST basta. Si se pasa a traduccion simultanea, se revisa.
- **Python (FastAPI):** buena opcion para IA, pero agrega otro lenguaje; la IA se consume por HTTP (Ollama, Anthropic), no se entrena aqui.
- **Java (Spring Boot):** robusto, pero mucho mas pesado para este alcance.

Consecuencias

Un solo lenguaje en todo el proyecto. Si crece, el modulo de traduccion se puede separar sin cambiar el contrato de la API.

ADR-003: SQLite como base de datos inicial

Fecha: 2026-10-14 · **Estado:** reemplazada por ADR-012 el 2026-10-15 (SQLite sigue como motor de las pruebas sin servidor) · **Responde a:** DRV-04, DRV-08

Contexto

Hace falta persistir usuarios, conversaciones y mensajes con integridad referencial, y que cualquiera pueda correr el proyecto sin instalar un servidor de base de datos.

Decision

SQLite con `node:sqlite` (incluido en Node 22) y solo consultas parametrizadas. El esquema sigue el modelo de dominio (`backend/src/adaptadores/persistencia/sqlite/esquema.js`). Las pruebas usan base en memoria.

Alternativas (comparadas al justificar)

- **PostgreSQL:** mejor para produccion y concurrencia; es el siguiente paso (F10).
- **MySQL:** similar a PostgreSQL, sin ventaja para este caso.
- **MongoDB:** no encaja con un modelo tan relacional (combinaciones unicas, relaciones muchos a muchos).
- **ORM (Prisma, Sequelize):** util con muchas entidades; con 11 tablas y consultas simples, las consultas parametrizadas cumplen la regla de seguridad sin otra dependencia.

Consecuencias

Instalacion cero. Para produccion hay que migrar a PostgreSQL; el SQL es estandar, asi que el cambio es acotado. `node:sqlite` aun es experimental en Node 22.

ADR-004: Traduccion con Qwen local, Claude Sonnet de respaldo y motor de reglas

Fecha: 2026-10-14 · **Estado:** aceptada · **Responde a:** DRV-01, DRV-02, DRV-05, DRV-06, DRV-08 · **Patron:** Circuit Breaker con degradacion aceptable

Contexto

El valor de TalkLate es traducir conservando tono, intencion y modismos; un traductor literal no lo logra. El equipo decidio usar **Qwen corriendo local** por ahora y volver a **Claude Sonnet** si no funciona.

Decision

Una fachada (`backend/src/aplicacion/traduccion.js`) prueba los motores en orden: **Qwen** servido por Ollama en el servidor de TalkLate, luego **Claude Sonnet** (API de Anthropic, si hay clave) y por ultimo un **motor de reglas** espanol/ingles con modismos y variantes por pais. Los dos modelos reciben el mismo prompt (equivalente cultural segun la region destino; devuelven tono, region de origen y nota cultural). Es el patron **Circuit Breaker** del catalogo: si un motor falla, se "abre" durante un minuto (no se le vuelve a llamar) y el sistema **degrada** al siguiente; el catalogo menciona justamente "traduccion literal como degradacion

aceptable", que es el papel del motor de reglas. No se usa Retry con backoff dentro de una misma peticion porque el usuario esta esperando: se prefiere pasar de inmediato al siguiente motor.

Alternativas (comparadas al justificar)

- **DeepL o Google Translate:** buena calidad literal, pero no explican ni adaptan modismos por region.
- **Solo Claude:** mejor calidad esperada, pero costo por mensaje, dependencia de internet y los mensajes salen a un tercero.
- **Solo motor de reglas:** gratis y rapido, pero no escala a mas idiomas ni a jerga nueva.
- **Otros modelos abiertos de traduccion (NLLB, Argos):** traducen frases, pero no siguen instrucciones de tono ni explican modismos.

Consecuencias

Qwen local no cuesta por mensaje y no envia datos a terceros (DRV-03, DRV-06), pero necesita un servidor con Ollama y el modelo descargado; su velocidad depende de ese equipo. Qwen corre en el **servidor**, no en el celular. La medicion real de tiempo y calidad se hace con `npm run medir` (DRV-01, DRV-02).

ADR-005: Voz: dictado en el cliente y expo-speech para la salida

Fecha: 2026-10-14 · **Estado:** reemplazada por ADR-014 (entrada) y ADR-015 (salida); `expo-speech` sigue como voz de respaldo · **Responde a:** DRV-07

Contexto

El usuario habla y escucha la traduccion. Transcribir audio en el servidor exige otro servicio y subir audio (dato personal).

Decision

El dictado se hace en el cliente (reconocimiento de voz del navegador en web; teclado del sistema en celular) y llega como texto con `canal: "voz"`. La traduccion se escucha con `expo-speech`, ajustando tono y velocidad segun la entonacion detectada.

Alternativas (comparadas al justificar)

- **Whisper u otro servicio de transcripcion en el servidor:** mas preciso y uniforme, y permitiria analizar el acento por el sonido; pero sube audio y requiere mas infraestructura.
- **Reconocimiento nativo dentro de la app:** requiere un development build de Expo.

Consecuencias

Sin costos extra y sin subir audio. En la practica el dictado del teclado no funciona de forma uniforme en Expo Go y no permite medir la voz; por eso ADR-014 lo reemplaza con grabacion en el celular y Whisper en el servidor.

ADR-006: Autenticacion con JWT de acceso corto y token de refresco rotativo

Fecha: 2026-10-14 · **Estado:** aceptada · **Responde a:** DRV-03 · **Patron:** Token-Based Authentication

Contexto

Usuario-RN-5 y RN-10, y la convencion de seguridad del proyecto: "tokens con expiracion (access corto, refresh mas largo)". El catalogo advierte que los JWT "no son revocables sin infraestructura adicional".

Decision

Token de acceso JWT de 15 minutos y token de refresco de 30 dias guardado solo como hash, que se rota en cada uso. Esa es la "infraestructura adicional": al cerrar sesion se revocan el token de acceso (lista de revocados) y el de refresco; al cambiar la contrasena se cierran las demas sesiones. Contrasenas con bcrypt, limite de intentos y codigo de recuperacion de un solo uso.

Alternativas (comparadas al justificar)

- **Sesiones en servidor:** revocacion trivial, pero guardan estado por sesion; JWT es lo que el catalogo recomienda para una app movil.
- **Proveedor externo (Firebase Auth, Auth0):** menos codigo propio, pero depende de un tercero y saca datos personales del sistema.

Consecuencias

La app renueva la sesion sola. El codigo de recuperacion sale por el puerto `Notificador`: en desarrollo se devuelve en la respuesta y en produccion lo envia un proveedor de correo conectado al mismo puerto.

ADR-007: Interfaz basada en el template Luna

Fecha: 2026-10-14 · **Estado:** aceptada

Contexto

El equipo adquirio el template Luna (chatbot) para no disenar la interfaz desde cero.

Decision

Se conservo su sistema de componentes y tema (claro y oscuro) y se reemplazaron sus vistas por las de TalkLate; se retiraron componentes y dependencias sin uso. El template original queda en `componentes/template/native-luna`.

Consecuencias

Interfaz consistente y rapida de construir. El mapeo etapa del journey -> pantalla esta en *documento 13 (Especificacion de vistas y UI)*.

ADR-008: Ajustes al modelo de dominio al implementarlo

Fecha: 2026-10-14 · **Estado:** aceptada · **Responde a:** DRV-04

Contexto

Al implementar aparecieron diferencias con el modelo de dominio enriquecido.

Decision

1. **Lenguaje - Region muchos a muchos** (`LenguajeRegion`): Conversacion-RN-1 pide validar que la region sea valida para el idioma, y un idioma se habla en varias regiones. Asi el nombre del lenguaje puede seguir siendo unico.
2. **Tablas de soporte** (`reporte`, `token_refresco`, `sesion_revocada`, `recuperacion`): no son objetos del modelo de dominio (los reportes son dinamicos y no tienen estructura fija), pero el sistema las necesita para funcionar.
3. **Usuario** gana `contrasena` (hash), `rol` y `consentimientoDatos`; **MensajeDestino.mensajeOriginal** pasa a ser referencia.

Consecuencias

documento 12 (Modelo de datos) refleja estos cambios y su correspondencia con la base de datos.

ADR-009: Identificadores del dominio en espanol (lenguaje ubicuo)

Fecha: 2026-10-14 · **Estado:** aceptada

Contexto

La regla transversal 1 de F0 pedia identificadores en ingles, pero todo el descubrimiento (modelo de dominio, reglas de negocio, historias) esta en espanol: Usuario, Conversacion, MensajeOriginal, Conversacion-RN-2...

Decision

Usar en el codigo los mismos nombres del modelo de dominio, en espanol (lenguaje ubicuo de Domain-Driven Design): tablas `conversacion`, `mensaje_original`; funciones `traducir`, `validarNombreConversacion`. Los terminos tecnicos y de librerias quedan en ingles. La regla de nombres de las convenciones del proyecto quedo actualizada.

Alternativas (comparadas al justificar)

- **Todo en ingles:** es la practica comun, pero obliga a traducir cada concepto y rompe la trazabilidad con el modelo de dominio y las reglas de negocio.

Consecuencias

Trazabilidad directa: una regla como Conversacion-RN-2 se encuentra en el codigo con el mismo nombre. Mezcla de idiomas en el codigo (dominio en espanol, framework en ingles). Excepcion (2026-10-15): el esquema de SurrealDB usa nombres en ingles para alinearse con el modelo de dominio en ingles; la traduccion de nombres queda encerrada en el adaptador (ver ADR-012).

ADR-010: Proteccion de datos personales

Fecha: 2026-10-14 · **Estado:** aceptada · **Responde a:** DRV-03 · **Patron:** Structured Logging

Contexto

TalkLate guarda conversaciones personales (PII segun la convencion de seguridad del proyecto) y opera en Colombia (Ley 1581 de 2012).

Decision

- Matriz de sensibilidad de datos (documento 16).
- Consentimiento obligatorio al registrarse y politica de tratamiento de datos.
- Eliminar la cuenta con todos sus datos (derecho al olvido).
- Contenido de mensajes **cifrado en reposo** con AES-256-GCM (clave por variable de entorno).
- Logs JSON que nunca guardan correo, contrasena, tokens, codigos ni contenido (patron Structured Logging, obligatorio en el catalogo).
- CORS con origenes explicitos y limites de uso, en especial sobre la traduccion.

Alternativas (comparadas al justificar)

- **Cifrar toda la base** (SQLCipher): protege todo, pero agrega una dependencia nativa; se deja para produccion junto con el cifrado del proveedor.

Consecuencias

Busquedas por contenido de mensajes no son posibles en la base (hoy no se necesitan). Sin la clave no se pueden leer los mensajes.

ADR-011: API v1 segun la convencion REST y documentada con OpenAPI

Fecha: 2026-10-14 · **Estado:** aceptada · **Patron:** Request-Reply

Contexto

La convencion REST del proyecto define versionado, formato de error, codigos, paginacion, health check y OpenAPI.

Decision

Base `/api/v1`; errores `{ statusCode, error, message, timestamp, path }`; `201` con `Location`; `204` al eliminar; `422` para reglas de negocio; listas `{ data, pagination }`; `GET /health`; contrato en `backend/openapi.yaml` con Swagger UI en `/api/v1/docs`. Los campos de las respuestas usan los nombres del dominio en snake_case (por ejemplo `fecha_inicio`) y `id`, para coincidir con las columnas (ADR-009); el ejemplo `identificador`/`fechaCreacion` de la convencion se toma como ilustrativo.

Consecuencias

Contrato claro para la app y para pruebas; cambios incompatibles van en `/api/v2`.

ADR-012: SurrealDB como fuente de verdad, detras de un repositorio

Fecha: 2026-10-15 · **Estado:** aceptada (decision cerrada el 2026-10-23) · **Reemplaza:** ADR-003 · **Responde a:** DRV-01, DRV-03, DRV-04, DRV-08

Contexto

ADR-003 dejo SQLite como base inicial y anoto que para produccion habia que pasar a un servidor de base de datos. Julian construyo el cambio (PR #29): el modelo de dominio tiene muchas relaciones (conversacion, lenguaje por conversacion, mensaje original y destino, reporte) y combinaciones unicas que se querian expresar directo en la base.

Decision

- **SurrealDB 3.x** es la fuente de verdad del backend. El esquema esta en `backend/db/surreal/schema.surql` (SCHEMAFULL) y se aplica al arrancar.
- Las relaciones son enlaces nativos (`record<...>`); las combinaciones unicas son indices `UNIQUE`; los valores permitidos (rol, estado, remitente) son `ASSERT` en el campo.
- El acceso a datos pasa por un **repositorio** con la misma interfaz para dos motores: `src/adaptadores/persistencia/surreal/repositorio.js` (ejecucion) y `src/adaptadores/persistencia/sqlite/repositorio.js` (pruebas sin servidor). Las rutas no conocen el motor; `DB_DRIVER` elige cual usar.

- La conexion es por HTTP (`/sql`) sin dependencias nuevas. Se entra una vez con `/signin` y se reutiliza el token; los valores se escriben como literales serializados (`lit`, `rid`), nunca concatenando texto del usuario.

Alternativas (comparadas al justificar)

- **Seguir con SQLite:** instalacion cero, pero es un archivo local; para varios usuarios a la vez y para produccion habia que cambiarla igual.
- **PostgreSQL:** muy maduro y era el paso previsto en ADR-003; las relaciones se modelan con llaves foraneas y JOIN. Se eligio SurrealDB porque los enlaces entre registros siguen el modelo de dominio sin tablas intermedias de mas y el esquema con ASSERT deja reglas del dominio en la base.
- **SDK oficial de SurrealDB:** mas comodo, pero agrega una dependencia; el endpoint HTTP basta para estas consultas.

Consecuencias

- Hace falta un servidor SurrealDB 3.x: el remoto para usar la app, o uno local para las pruebas (`backend/README-surreal.md`). Con la 2.x el esquema no carga; el backend lo revisa al arrancar.
- Mandar usuario y clave en cada consulta costaba unos 35 ms por consulta (unos 420 ms por mensaje); con el token de sesion bajo a unos 28 ms por mensaje. Sigue por encima de SQLite (unos 7 ms), pero muy lejos del limite de 3 s de DRV-01.
- La renovacion de la sesion revoca el token de refresco en un solo paso (`tokens.consumir`), porque con un servidor dos pedidos pueden llegar al mismo tiempo.
- CI corre la suite de la API contra los dos motores (job `backend-surreal`).
- En produccion son obligatorias `SURREAL_USER` y `SURREAL_PASS` (no se acepta root/root).
- Algunas consultas filtran recorriendo enlaces (por ejemplo, los mensajes de una conversacion); al crecer el volumen se agregan indices sobre esos campos.

Evaluacion frente a PostgreSQL

Se comparo SurrealDB con PostgreSQL contra lo que pide TalkLate y se eligio **SurrealDB**. Las razones:

1. **El dominio es una cadena de relaciones y SurrealDB las guarda como enlaces nativos.** Conversacion -> idioma de la conversacion -> mensaje original -> mensaje traducido -> reporte. Cada campo `record<...>` apunta directo al registro relacionado, y una consulta lo recorre sin JOIN. Ejemplo real del adaptador: para saber si una traduccion es del usuario se escribe `mensaje_original.lenguaje_conversacion.conversacion.usuario = $usuario` en una sola expresion (REQ-CONV-007, REQ-REP-001).
2. **Las reglas del dominio tambien viven en la base (defensa en profundidad).** El esquema es SCHEMAFULL: las combinaciones unicas del modelo de datos son indices `UNIQUE` (correo, nombre y codigo de idioma, region dentro del pais, un reporte por traduccion y usuario); los valores permitidos son `ASSERT` (rol, estado del reporte, remitente, estado de entrega); y hay validaciones de formato y longitud (correo con `string::is_email`, nombre de conversacion de 3 a 80 caracteres). Si un error en la API dejara pasar un dato invalido, la base lo rechaza.

3. **Es multimodelo, y eso encaja con lo que viene sin agregar otra base.** Los reportes son dinamicos (sin estructura fija): SurrealDB admite campos flexibles junto a tablas estrictas. La evolucion prevista, usar los reportes y un glosario de modismos para mejorar la traduccion, puede apoyarse en sus indices vectoriales (HNSW) y en relaciones de grafo (`RELATE`) para equivalencias entre regiones, todo en la misma base.
4. **Gestionada y con control de acceso fino.** SurrealDB Cloud evita instalar y operar un servidor de base de datos, algo importante para un equipo de dos personas. SurrealDB permite crear usuarios con permisos limitados a un namespace o a una base; el backend usa un usuario de base propio, limitado a `talklate/app`, y no el root (minimo privilegio, ADR-010).
5. **Ya esta construida y probada.** Funciona contra una instancia real y CI corre la suite de la API contra SurrealDB en cada cambio.

Lo que se acepta a cambio (y como se maneja):

- Es mas joven que PostgreSQL y su comunidad es menor. Se usa la version 3.x fija en CI, y el acceso a datos queda aislado detras del puerto `Repositorio` (ADR-013): si algun dia hubiera que cambiar de base, seria un adaptador nuevo y no una reescritura.
- No se usan consultas preparadas como en SQL: el adaptador nunca concatena texto del usuario; todo valor pasa por `lit`/`rid` y hay pruebas de inyeccion (comillas, saltos de linea, `"; DELETE ...`).
- Las consultas que recorren enlaces no usan indices: se revisan con mas datos y se agregan indices donde haga falta.

Sintesis: "Elegimos SurrealDB porque nuestro dominio es una cadena de relaciones (conversacion, mensajes, traducciones, reportes) que guarda como enlaces nativos, porque nos deja poner las reglas del dominio en el propio esquema y porque, al ser multimodelo, soporta lo que viene (reportes flexibles, busqueda vectorial de modismos) sin agregar otra base. Su riesgo, ser una tecnologia mas joven, lo controlamos con la arquitectura hexagonal: la base esta detras de un puerto y se puede cambiar sin tocar las reglas."

Esquema e instancia

- **Esquema en ingles:** las tablas y campos de SurrealDB estan en ingles (`user`, `conversation`, `original_message`, `translated_message`...), alineados con el modelo de dominio en ingles. Es una excepcion a ADR-009 solo para la base: la API, el codigo y los valores de los enumerados (`origen`/`destino`, `abierto`...) siguen en espanol, y el adaptador (`src/adaptadores/persistencia/surreal/repositorio.js`) traduce los nombres en cada consulta.
- **Instancia remota:** la app usa por defecto una instancia de SurrealDB Cloud; las credenciales van en `backend/.env` (fuera del repo). Las pruebas y CI usan un SurrealDB local para no crear ni borrar datos en el remoto.

ADR-013: Arquitectura hexagonal (puertos y adaptadores) dentro del monolito modular

Fecha: 2026-10-23 · **Estado:** aceptada · **Complementa:** ADR-002, ADR-004, ADR-012 · **Responde a:** DRV-04, DRV-05, DRV-08

Contexto

Una revision de la arquitectura confirmo que el monolito modular es la decision correcta (ADR-002), pero mostro que su modularidad no era explicita: las rutas HTTP mezclaban reglas de negocio, acceso a datos y cifrado, y la fachada de traduccion conocia cada motor por su nombre. La pregunta "y si manana crece?" debe poder responderse sin partir el sistema en microservicios.

Decision

Organizar la API en capas de **puertos y adaptadores** (Alistair Cockburn), sin cambiar su contrato:

- **Dominio** (`src/dominio`): validaciones, errores de negocio, analisis de texto, prosodia de la voz y expresion de cada tono. No depende de nada externo.
- **Aplicacion** (`src/aplicacion`): un caso de uso por epica (cuentas, sesiones, conversaciones, traduccion, voz, catalogo, reportes). Solo importa el dominio y los puertos.
- **Puertos** (`src/puertos`): contratos `Repositorio`, `MotorTraduccion`, `Transcriptor`, `Sintetizador`, `Notificador`, `Cifrador`, `Hasheador` y `FirmadorTokens`, verificados al arrancar.
- **Adaptadores** (`src/adaptadores`): de entrada, HTTP con Express; de salida, SurrealDB y SQLite, Qwen, Claude y reglas, Whisper, VoiceStudio, Azure y OpenAI, correo y AES-GCM.
- **Composicion** (`src/composicion.js`): unico lugar que conoce los adaptadores concretos; los elige segun el entorno y arma los casos de uso.

El subsistema de traduccion queda detras del puerto `MotorTraduccion`: el caso de uso aplica el Circuit Breaker (ADR-004) sobre cualquier lista de motores, sin saber cuales son.

Alternativas (comparadas al justificar)

- **Dejar el monolito como estaba:** funciona, pero las reglas viven en las rutas y cambiar un motor o la base obliga a tocar varios archivos.
- **Microservicios (traduccion aparte):** limites claros, pero agrega red, despliegue y fallas distribuidas que un equipo de dos no necesita (ADR-002).
- **Capas clasicas (controlador, servicio, repositorio) sin puertos:** separa responsabilidades, pero el servicio sigue dependiendo de implementaciones concretas.
- **Clean Architecture completa con un framework de inyeccion de dependencias:** mas ceremonia y dependencias para el mismo resultado; aqui la composicion manual basta.

Consecuencias

- Limites claros sin el costo distribuido: cambiar de base de datos (por ejemplo a PostgreSQL) o agregar un motor es escribir un adaptador nuevo; los casos de uso no cambian.
- El caso de uso de traduccion y el Circuit Breaker se prueban con motores falsos, sin red ni HTTP (`test/motores.test.js`).
- Un adaptador incompleto falla al arrancar (`asegurarPuerto`), no a mitad de una peticion.
- Mas archivos y una indireccion mas. El contrato de la API, la base y las pruebas de punta a punta no cambiaron (las mismas 16 pruebas de la API pasan con SQLite y con SurrealDB).
- Los nombres siguen ADR-009 (dominio en espanol).
- Las reglas de dependencia entre capas se verifican con dependency-cruiser (`backend/.dependency-cruiser.cjs`, `npm run arch:check`): dominio puro, aplicacion sin adaptadores, puertos como contratos y adaptadores que no se cruzan entre categorias. El chequeo corre en el CI en cada pull request y hoy no marca ninguna violacion: el hash de contrasenas (bcrypt) y los tokens (JWT) tambien entran por puertos (`Hasheador` y `FirmadorTokens`).

ADR-014: Voz en Expo Go con grabacion en el celular, Whisper en el servidor y entonacion por texto y voz

Estado: Aceptada (23/10/2026). Reemplaza la parte de ADR-005 que usaba el dictado del teclado.

Contexto: Expo Go no trae reconocimiento de voz nativo, y el dictado del teclado no funciona de forma uniforme ni permite medir la voz. Ademas, la traduccion debe sonar con la misma entonacion de quien habla.

Decision:

- La app graba con `expo-audio` (funciona en Expo Go) y mide el volumen cada 100 ms.
- El audio va en base64 a `POST /api/v1/conversaciones/:id/mensajes/voz`. El nuevo puerto `Transcriptor` lo pasa a texto con un adaptador Whisper (API compatible con OpenAI `/v1/audio/transcriptions`: faster-whisper/speaches o whisper.cpp locales). Variables: `TRANSCRIPCION_URL`, `TRANSCRIPCION_MODELO`.
- `dominio/prosodia.js` saca el tono de la voz (volumen, variacion y palabras por segundo). El tono del texto manda salvo que sea Neutral; la descripcion de la prosodia va al prompt del LLM y `entrega.ritmo_voz` ajusta la velocidad de la voz de salida.
- La interfaz tiene dos modos, como ChatGPT: la burbuja animada del template Luna (AiCircle) y el chat. El modo se recuerda en el celular.

Consecuencias: Se necesita un servidor Whisper (se recomienda el modelo `small`; `tiny` confunde jerga como "parce"). Sin `TRANSCRIPCION_URL` el endpoint de voz responde 503 y el texto sigue funcionando.

ADR-015: Voz natural con emocion en el servidor (puerto Sintetizador)

Estado: Aceptada (26/10/2026). Complementa ADR-014.

Contexto: La voz del celular (expo-speech) suena robotica y casi no cambia con la emocion; subir o bajar el tono no alcanza para que se note enojo o alegria.

Decision:

- Nuevo puerto `Sintetizador` con dos adaptadores, elegidos con `VOZ_MOTOR` (auto, azure, openai, ninguno):
  - **Azure Speech** (voces neuronales, capa gratis de 500 mil caracteres al mes). La emocion va en SSML con `mstts:express-as` (angry, cheerful, sad, friendly) mas prosodia de respaldo para voces sin estilos. La voz se elige por idioma y pais (ej. es-CO-SalomeNeural, en-US-JennyNeural).
  - **API compatible con OpenAI** (`/v1/audio/speech`). Con `gpt-4o-mini-tts` la emocion y el acento van como instruccion en lenguaje natural; tambien sirve un Kokoro-FastAPI local.
- `dominio/voz.js` define como suena cada tono (estilo, prosodia e instruccion); los adaptadores solo lo traducen a su API.
- `POST /api/v1/voz` devuelve una URL de mp3 que vence en 5 minutos (`GET /voz/audio/:id`, id aleatorio de 128 bits), para que el reproductor del celular la abra sin guardar archivos.
- Si no hay motor configurado o falla, la app vuelve a la voz del celular con tono y velocidad ajustados.

Consecuencias: La voz suena natural y con emocion cuando hay clave de Azure o de OpenAI; sin ellas todo sigue funcionando con la voz del celular. Los audios viven solo en memoria del servidor (no se guardan).

ADR-016: VoiceStudio local como primer motor de voz con emocion

Estado: Aceptada (29/10/2026). Extiende ADR-015.

Contexto: Azure y OpenAI dan voz natural, pero cobran o piden clave y salen a internet. El proyecto ya traduce con Qwen local (ADR-004), y una voz local mantiene la misma linea: sin costo por uso y sin enviar la voz a terceros. VoiceStudio (github.com/debpalash/VoiceStudio) reune motores de voz locales; su motor IndexTTS 2 acepta la emocion como un vector de 8 pesos y habla espanol e ingles.

Decision:

- Nuevo adaptador `adaptadores/sintesis/voicestudio.js` del puerto `Sintetizador`.
  - Con IndexTTS 2 usa `POST /generate` con `emo_vector`. Con los otros motores usa `POST /v1/audio/speech` (compatible con OpenAI) y manda la emocion como instruccion en texto.
- Con `VOZ_MOTOR=auto` el orden es VoiceStudio, luego Azure, luego OpenAI, y al final la voz del celular.
- El equipo de despliegue de referencia es una Mac con chip M5 y 32 GB de RAM, con instalacion nativa (Metal/MLX). En un equipo Intel sin GPU NVIDIA, IndexTTS en CPU tarda decenas de segundos por frase; ahi el primer motor es Azure (`VOZ_MOTOR=azure`).
- La guia de montaje y medicion esta en `backend/VOICESTUDIO_MAC.md` (`npm run medir:voz`).

Consecuencias:

- Voz con emocion gratis y local, y la posibilidad de clonar una voz con permiso.
- La latencia se mide con `npm run medir:voz` en el equipo de despliegue (meta: mediana de 5 s o menos por frase, DRV-01).

- La licencia de VoiceStudio es AGPL-3.0 y cada modelo tiene la suya: es valida para el uso academico; para un uso comercial se usa Azure u OpenAI detras del mismo puerto.
- El adaptador queda cubierto por pruebas automaticas de las dos rutas (`/generate` y `/v1/audio/speech`), del vector de emocion y de los errores.

ADR-017: Piramide de pruebas en tres niveles, todos en el CI

Estado: aceptada (05/11/2026). **Responde a:** DRV-08, DRV-03.

Contexto: Las pruebas del backend estaban en cuatro archivos mezclados (reglas, adaptadores e integracion) y no habia pruebas de la app real en un navegador. No se veia que nivel cubria que, ni cuanto costaba correr cada uno.

Decision:

- Organizar las pruebas como piramide: **base** de unitarias (dominio, casos de uso y adaptadores con dobles; `backend/test/unitarias/` y `frontend/test/`), **medio** de integracion (API HTTP con base real; `backend/test/integracion/`, contra SQLite y SurrealDB) y **punta** de punta a punta (servidor real como proceso en `backend/test/e2e/` y app web con Playwright en `e2e/`).
- Un script por nivel (`test:unitarias`, `test:integracion`, `test:e2e`) y un paso por nivel en el CI; la app web corre en un job aparte (`e2e-web`) que solo arranca si los niveles de abajo pasaron.
- Cobertura minima de 80 % en unitarias + integracion, y `arch:check` (dependency-cruiser) en el CI. Para que pase, bcrypt y jsonwebtoken entran por los puertos `Hasheador` y `FirmadorTokens`.
- En las pruebas de punta a punta, los servicios de IA (Whisper, VoiceStudio) se reemplazan por servidores HTTP falsos que hablan su mismo protocolo: se prueban los adaptadores reales sin depender de modelos pesados.

Alternativas:

- **Jest o Vitest en el backend:** mas funciones, pero `node:test` ya viene con Node 22, no agrega dependencias y tiene cobertura incluida.
- **Cypress para la web:** maduro, pero Playwright es mas liviano en CI, emula celulares y ya era la herramienta usada para revisar la app.
- **Detox o Maestro en el celular:** prueban la app nativa, pero exigen emuladores en CI; la app web comparte el mismo codigo (ADR-001) y cubre el recorrido.

Consecuencias: 51 unitarias, 18 de integracion (x2 bases) y 4 de punta a punta; cobertura de lineas de 96.6 %. Cada falla dice en que nivel esta el problema. Detalle en `calidad/piramide_de_pruebas.md`.

ADR-018: Escaneo de seguridad con Trivy en el CI

Estado: aceptada (05/11/2026). **Responde a:** DRV-03.

Contexto: TalkLate maneja datos personales (Ley 1581) y depende de cientos de paquetes de npm. Una dependencia vulnerable o una clave escrita en el codigo son riesgos que las pruebas funcionales no detectan.

Decision:

- Escanear el repositorio con Trivy: dependencias npm (de produccion y de desarrollo), secretos y configuraciones inseguras. Bloquea el merge si hay hallazgos HIGH o CRITICAL.
- Corre en cada pull request, en cada push a `develop` y `main` y cada lunes (los CVE nuevos aparecen aunque el codigo no cambie).
- Trivy se instala desde el binario oficial con version fija (0.69.3) y checksum SHA-256 verificado. No se usan `aquasecurity/trivy-action` ni `setup-trivy`, comprometidas en marzo de 2026.
- Las vulnerabilidades indirectas se corrigen con `overrides` de npm dentro de la misma version mayor. Lo que no tiene parche compatible va a `.trivyignore` con su razon, su riesgo real y una fecha de vencimiento.

Alternativas:

- **`npm audit`:** solo revisa npm y no busca secretos ni configuraciones; ademas no permite excepciones con vencimiento.
- **Dependabot o Snyk:** utiles, pero dependen de la configuracion de la organizacion en GitHub o de un servicio externo; Trivy corre igual en local y en CI.

Consecuencias: Se corrigieron 1 HIGH en el backend y 1 CRITICAL + 50 HIGH en el frontend (produccion y desarrollo). Queda 1 excepcion documentada (`decode-uri-component`, vence el 28/02/2027). Detalle en `calidad/seguridad_trivy.md`.